StoryDNA — catch “the same story under a new name” before a microdrama is produced

Kuku AI Generalist Assignment · **Problem 3 (own problem)** · Uday Rahul Ingle · Roll 230102100 · IIT Guwahati

1. Problem, who has it, how I know

User & step: a creative-development editor doing the *first review of a new pitch, before any episode is written or generated*. **Bottleneck:** Kuku reportedly releases 150–250 original shows a month (Indian Television; Hollywood Reporter India). Nobody can remember every earlier plot, and with AI generation, plots drift toward the same few chains (hidden billionaire → humiliation → public reveal). The editor needs one question answered fast: **“Is this pitch the same *story* as something we already made, or just the same *genre*?”** and the evidence to check it.

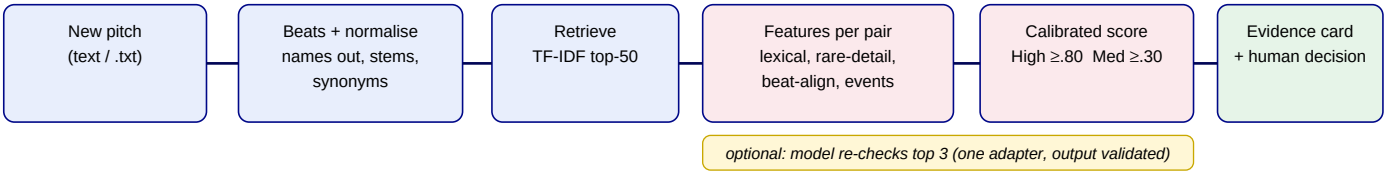
Evidence it is real (secondary only): Redseer’s Dec-2025 India consumer research names repetitive storylines and slow refresh among the main churn drivers (via Exchange4media, Storyboard18); Meta–Ormax 2026 flags creative fatigue from repeated tropes (Exchange4media); China’s regulator told micro-drama makers to drop stock tropes (Sixth Tone, 2024). **What I did not do:** I did not interview viewers or Kuku editors, and I cannot measure Kuku’s actual redundancy rate. The claim “this matters more than Problems 1 and 2” is therefore a **hypothesis**: Problem 1 optimises marketing of a finished show, whereas a redundant premise wastes spend across every episode, but I have no Kuku numbers to prove it.

Kuku already has related tech. It publicly describes an “AI Storytelling Stack” (Business Standard, Sep 2026), so I assume retrieval over stories exists internally. I did not try to out-build it. I tested the narrower, checkable question: can a cheap, explainable pipeline **separate same-plot from same-trope and show its evidence**?

2. Assumptions (the brief was silent)

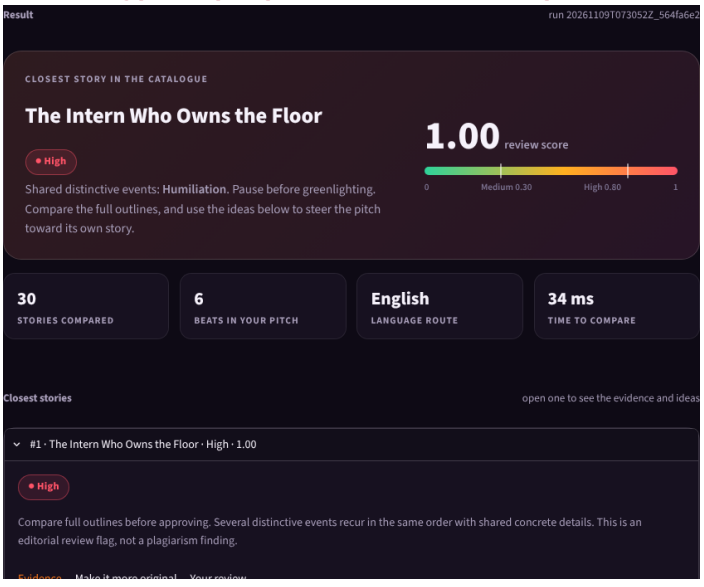
- **A1** Input is a pitch/synopsis of 4–12 beats, not a full screenplay. Long scripts would first be condensed to beats (optional model step, built, tested on mocks only).
- **A2** The Kuku input pack is one show, so it cannot test cross-show matching, and it is confidential. **None of it is used or in the repo.**
- **A3** Catalogue = **30 original story cards written by an AI assistant for this assignment** (typical microdrama patterns, not real shows, no scraped text). Results say nothing about Kuku’s catalogue.
- **A4** Output is an *editorial review priority*, never a plagiarism or legal verdict; ideas and tropes are not protectable. A human always decides; nothing is auto-rejected. The matcher reads English; Hindi/Hinglish go through an offline glossary, other languages need a model (see failures).

3. How it works — what is code, what is a model, where the human sits



Code (default, free, offline): splitting into beats; masking character names so renamed copies still match; stemming and synonym folding; TF-IDF retrieval; six per-pair features (lexical cosine, rare-detail overlap, ordered beat alignment, ordered/rarity-weighted event overlap from a 31-event rule lexicon, plain TF-IDF); a small logistic combination fitted on 54 dev pitches; tiers; schema/evidence validation. **Model (optional, one adapter):** re-checks the top 3 and condenses long text. Provider and model are env-vars (Anthropic or any OpenAI-compatible endpoint), prompts are plain files, and **every model answer is validated**: cited beats must exist, claims need a reason, “same plot” with no valid evidence is downgraded to “unverified”, and a model/heuristic disagreement is flagged to the human, never silently resolved. **Human:** sees the matched beats side by side, the generic-trope events separated from distinctive ones, and the beats that differ, then marks *needs review / false alarm / safe*. High/Medium matches also get ideas for making the pitch more original (a what-if re-score, beat-level prompts); rewriting the two flagged sample beats took it from High to below High.

4. Prototype output (real run on an unseen pitch I wrote; not in the catalogue)



The pitch is a hotel-heiress story that shares its skeleton with catalogue card *The Intern Who Owns the Floor* and a framing beat with *Night Shift Owner*. The tool returns S05 as **High** and S02 as **Medium**, labels PUBLIC_REVEAL and COMEUPPANCE as *trope-only*, and lists the beats that differ. Honest miss: the tagger did not recognise “hides her family name” as concealed identity, so one distinctive event is missing from the evidence.

Run it again: paste any new pitch, upload a .txt, or swap the catalogue for your own CSV/JSON (`story_id,title,synopsis`). CLI: `python -m storydna compare --pitch new.txt`. Exports JSON/CSV with the reviewer’s decision. About 35 ms per pitch including the what-if preview, no API key needed.

Prototype, raw outputs, tests and README are in the repo submitted with this form. Page 2: how I know it works, and where it fails.

5. Quality checking — what I ran, in order (raw files: `raw_outputs/`)

I wrote each test set **before** running the version it tests, hash-froze it, and kept every raw report including bad ones. Positives = a pitch whose plot matches one catalogue story (exact copy, renamed paraphrase, partial). Negatives = same genre/trope but different plot, or unrelated. "False High" = a negative rated High.

Round (what changed)	Pitches	Positives: right story ranked #1	Positives rated High	Negatives rated High / Medium+	Reading
1 — a-priori rules	40	23/23 (plain TF-IDF 23/23)	16/23 (exact 5/8, paraphrase 8/10, partial 3/5)	0/17 / **15/17**	Medium far too loose; punctuation-free text collapsed to one beat (bug, 3 exact copies missed).
2 — bug fix + tighter rules, tuned on the same 40	40	23/23	22/23	0/17 / 2/17	**In-sample, so proves nothing.**
3 — 14 fresh pitches, new vocabulary, run once on round-2 tool	14	**6/7** (plain TF-IDF 7/7)	2/5 paraphrases	**2/7 false High** / 2/7	**Overfit.** My hand-built event lexicon failed to generalise; plain cosine beat it (AUC .91 vs .69).
Fix: name-detection bug (it deleted "Billionaire", "Widow"), lexicon-free beat alignment, weights fitted with leave-one-out CV on rounds 1+3	54 (CV)	30/30 (hit@1 100%)	—	AUC overlap vs trope-only **1.00** (plain TF-IDF 0.96; fixed weights 0.94)	Thin margin: worst non-overlap 0.64 vs weakest true overlap 0.68, so High set conservatively at 0.80.
4 — final test: 16 new pitches, scorer hash-frozen, run once	16	**7/7** (plain TF-IDF 7/7)	**7/7**	**0/9 / 2/9**	Good, but n is tiny and plain TF-IDF is equally perfect here.

Honest reading. On this small, easy corpus plain TF-IDF already finds the right story; StoryDNA's measured gain is in *separating overlap from trope-only* (cross-validated AUC 1.00 vs 0.96) and in the evidence it prints, not in retrieval. Test sets and catalogue come from the same author (an AI assistant), so all numbers are optimistic. A real test needs Kuku's catalogue and editor-labelled pairs.

Where it fails (all in the raw outputs)

- **Near-miss false positive:** an unrelated chess-championship pitch scored 0.64 (cross-validated) against the maths-olympiad card (shared "competition → final" events). Round 3 also rated a blackmail/twins trope pitch High.
- **Language:** at first Hinglish returned a silent Low. I then added a language layer (detection, offline Hindi/Hinglish glossary, optional model translation). On 22 frozen Hindi/Hinglish runs (16 true overlaps, 6 not) it ranked the right story first as High 16/16 (layer off: 8/16) with 0/6 false Highs, but the glossary and pitches share one AI author, so this is optimistic, and glossed text is crude. Other languages: reported "not assessed" unless a model is configured; model translation is mock-tested only.
- **Heavy paraphrase** ("becomes the cleaner on the late shift" for "night janitor") relies on lexical overlap; the model step is meant for this and is **untested against a live API** (mock-tested only, incl. a mock that invents evidence).
- Short pitches (<4 beats) and synopsis-only cards carry little structure; the tool says so in a warning. 16 malformed-input cases: 0 crashes, readable errors.

AI tools used, and where they got it wrong

Tools: Claude (Anthropic) for code, test design, research checks and drafting; scikit-learn, Streamlit. **Mistakes I caught:** (1) the first scorer the AI wrote scored 100% on the set it was tuned on and then fell apart on 14 fresh pitches (round 3), so I rebuilt it and froze a new test; (2) its name-masking silently removed real words and zeroed one story's lexical score; (3) the research brief cited a "54 of 690 complaints" statistic I could not verify, so I left it out.

6. Scale, cost, and how a real team would know it worked

Catalogue size	30	300	3,000	10,000
Index build / median query / memory	0.05 s / 4 ms / 171 MB	0.3 s / 6 ms / 175 MB	3.1 s / 8 ms / 212 MB	10.3 s / 11 ms / 327 MB

Measured on a 2-core sandbox with synthetic recombined cards: cost only, not accuracy. Retrieval stays O(N) with a 50-candidate shortlist; beyond ~100k stories, swap in a vector index. Baseline API cost is **zero**. Optional model step: top-3 × about 550 input + 200 output tokens (estimated) ≈ 2.3k tokens per pitch, i.e. tokens × the chosen provider's rate; I did not invent a rupee figure.

Pilot (10–20 real pitches, two weeks): an editor reviews half with the old process and half with StoryDNA (order randomised). Track **review minutes per pitch**, **false alerts per pitch** (target ≤ 1), precision of High flags as judged by two editors, editor agreement with the tiers, and whether editors find the original card faster. Success = fewer minutes without missing a redundancy the editors later confirm. **Not claimed:** retention, revenue or legal-risk improvement.